

Agentic AI Security Supervisor for Information Systems

Semester Project Report – Submitted in Partial Fulfillment of the Requirements for the
Degree of Bachelor in
Artificial Intelligence Engineering – Intelligent Information Security Systems Engineering

Prepared by:

Omar Tanah

Louay Almarri

Hamzeh Beirouti

Supervised by:

Dr. Christine Zenieh

Eng. Ahmad Shekha

Academic Year 2025-2026

Agentic AI Security Supervisor for Information Systems

تقرير مشروع فصلي - هندسة الذكاء الاصطناعي - هندسة نظم أمن المعلومات الذكية

إعداد:

عمر طانه

لؤي المعري

حمرة بيروتي

إشراف:

د. كريستين زينية

م. أحمد شيخة

العام الدراسي 2025-2026

List of Abbreviations and Scientific Terms.....	4
Abstract.....	6
المُلخَص.....	7
Introduction.....	8
.....	11
Chapter 1: Theoretical Framework.....	11
1.1 Introduction.....	12
This chapter establishes the theoretical foundation for the proposed Agentic AI Security Supervisor. It defines Agentic AI and its core characteristics, explains the ReAct (Reasoning + Acting) framework that drives the agent’s decision-making, and discusses the risks and challenges associated with deploying autonomous AI agents in cybersecurity.....	12
1.2 Agentic AI: Concept and Characteristics	12
1.3 The ReAct Framework: Mechanism and Operation	13
1.4 Risks and Challenges of Agentic AI in Cybersecurity	14
1.5 Mitigating Agentic AI Risks with the ReAct Framework.....	17
1.6 Conclusion.....	18
Chapter 2: Literature Review and Research Gap	19
2.1 Introduction.....	21
2.2 Current State of Security Automation	21
2.3 LLMs in Cybersecurity.....	22
2.4 SIEM Systems and the Triage Process	23
2.5 Recent Study on LLM-SIEM Integration	24
2.6 Strategic Integration: LLM as a Security Supervisor over SIEM.....	24
2.7 The Emergence of Agentic AI.....	25
2.7.1 Agentic AI in Security Operations Centers.....	26
2.7.2 Why a Single ReAct Agent Beats Multi-Agent Systems.....	26

2.8 Synthesizing the Gap: What Existing Systems Miss.....	27
2.8.1 The Gap in Autonomous Response (CORTEX & Others).....	30
2.9 Distinction from Existing Security Technologies	31
2.10 Research Question.....	31
2.11 Conclusion.....	31
Chapter 3: Proposed Solution	32
3.1 Introduction.....	33
3.2 Proposed Solution.....	34
3.3 System Architecture Overview.....	34
3.4 Alert Ingestion and Normalization Layer.....	35
3.5 Autonomous Triage Engine.....	36
3.6 The ReAct Agent Core.....	40
3.7 Perception Layer (Tools).....	40
3.8 Correlation Engine.....	42
3.9 Incident Report and Audit Trail.....	42
3.10 Reasoning Layer (LLM Core).....	43
3.11 Scope of Supervision	43
Chapter 4: Practical Application and Results.....	46
4.1 Introduction.....	47
4.2 Challenges of Integrating AI with Cybersecurity	47
4.3 Python in Cybersecurity.....	47
4.4 Virtual Environment Setup using VirtualBox.....	48
4.4.1 Internal Network Communication (NAT & Host-Only):	48
4.4.2 Virtual Environment Components:	49
4.5 Libraries and Artificial Intelligence Technologies Used.....	50
4.6 Wazuh platform as an open-source SIEM system	51
4.7 Large Language Model GPT-OSS and Running it via Ollama	52

4.8 System Overview and Implementation.....	54
4.9 Scenario 1: SSH Brute-Force Attack — Autonomous Containment.....	54
4.10 Scenario 2: Escalation and Aggregated Alert Handling.....	55
4.11 Discussion of Results.....	56
4.12 Conclusion.....	57
Conclusion	58
References.....	60

List of Abbreviations and Scientific Terms

Abbreviation / Term	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CLIPS	C Language Integrated Production System
CNN	Convolutional Neural Network
CTI	Cyber Threat Intelligence
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
LLM	Large Language Model
LSTM	Long Short-Term Memory
MITRE ATT&CK	Adversarial Tactics, Techniques, and Common Knowledge
MTTR	Mean Time to Respond/Recover
RAG	Retrieval-Augmented Generation
ReAct	Reasoning + Acting
SIEM	Security Information and Event Management
SOAR	Security Orchestration, Automation, and Response
SOC	Security Operations Center

Wazuh	Open-source SIEM platform
-------	---------------------------

Abstract

Security Operations Centers (SOCs) rely on Tier-1 analysts to manually triage thousands of daily alerts, a repetitive workload that causes alert fatigue and delayed response. This project proposes an Agentic AI system that functions as an autonomous Tier-1 SOC analyst, replicating the human tasks of monitoring a SIEM, gathering context, reasoning about alerts, and executing responses such as blocking malicious IPs. The objective is to automate the full analyst workflow – triage, investigation, decision, response, and escalation – while maintaining a complete audit trail. The system integrates with the Wazuh SIEM platform and uses a ReAct (Reasoning + Acting) framework powered by a locally hosted GPT-OSS large language model. It receives alerts via a Flask API, enriches them with threat intelligence, calculates false-positive probability and priority scores, then iteratively decides and executes responses, with every action logged to an immutable SQLite database. Tested in a virtual lab (Ubuntu, Windows, Kali), the agent autonomously blocks SSH brute-force attacks across both Linux and Windows within 60 seconds – matching human speed while eliminating manual error. When a compromise is confirmed, it escalates to a human Tier-2 analyst with a structured ticket and a complete reasoning trace. The system successfully handles the full Tier-1 analyst workload with full transparency.

Keywords: Agentic AI, Tier-1 SOC analyst, autonomous incident response, ReAct, SIEM, Wazuh.

الملخص

تعتمد مراكز عمليات الأمن (SOCs) على محلي المستوى الأول لفرز آلاف التنبيهات يوميًا يدويًا، وهو عبء عمل متكرر يسبب إرهاق المحللين وتأخر الاستجابة. يقترح هذا المشروع نظام ذكاء اصطناعي فعال يعمل كمحلل مستقل من المستوى الأول في مركز عمليات الأمن، حيث يحاكي المهام البشرية المتمثلة في مراقبة منصة SIEM، وجمع السياق، والتفكير في التنبيهات، وتنفيذ الاستجابات مثل حجب عناوين IP الضارة. الهدف هو أتمتة سير عمل المحلل بالكامل – الفرز، التحقيق، القرار، الاستجابة، والتصعيد – مع الحفاظ على مسار تدقيق كامل. يتكامل النظام مع منصة Wazuh لإدارة معلومات وأحداث الأمن (SIEM) ويستخدم إطار ReAct (التفكير + الفعل) المدعوم بنموذج لغة كبير GPT-OSS يعمل محليًا. يستقبل التنبيهات عبر واجهة Flask API، ويثرىها باستخبارات التهديدات، ويحسب احتمالية الإيجابية الخاطئة ودرجات الأولوية، ثم يقرر وينفذ الاستجابات بشكل تكراري، مع تسجيل كل إجراء في قاعدة بيانات SQLite غير قابلة للتغيير. تم اختباره في مختبر افتراضي (Kali و Windows و Ubuntu)، حيث يقوم العامل بشكل ذاتي بحجب هجمات القوة العمياء على SSH عبر منصتي Linux و Windows خلال 60 ثانية – وهو ما يعادل سرعة المحلل البشري مع القضاء على الخطأ اليدوي. عند تأكد الاختراق، يقوم بتصعيد الحادث إلى محلل بشري من المستوى الثاني مع تذكرة منظمة وأثر تفكير كامل. يدير النظام بنجاح عبء العمل الكامل لمحلل المستوى الأول بشفافية كاملة.

الكلمات المفتاحية: وكيل الذكاء الاصطناعي، محلل المستوى الأول في مركز عمليات الأمن، الاستجابة التلقائية للحوادث، Wazuh ، SIEM ، ReAct.

Introduction

Security Operations Centers (SOCs) are the first line of defense against cyber threats, yet they face an operational crisis. A typical mid-size SOC generates thousands of security alerts every day, with false-positive rates approaching 99% in many environments [3]. The responsibility for initial triage falls entirely on Tier-1 analysts – junior security professionals who manually examine each alert, query multiple tools for context (IP reputation, user history, asset criticality), follow static playbooks, and decide whether to close the alert as a false positive or escalate it to a Tier-2 analyst. Studies have shown that 30-50% of a Tier-1 analyst’s time is consumed by repetitive triage tasks, leading to alert fatigue, burnout, and delayed response to genuine threats [7]. The global shortage of skilled cybersecurity personnel exacerbates the problem, leaving even well-staffed SOCs struggling to keep pace with the volume and sophistication of modern attacks [1].

The research problem addressed by this project is the fundamental mismatch between human analytical capacity and the ever-increasing volume of security alerts. Traditional automation tools, such as Security Orchestration, Automation, and Response (SOAR) platforms, attempt to reduce the burden by executing predefined actions when specific conditions are met. However, these systems operate on rigid, rule-based playbooks that lack contextual awareness [3]. A SOAR system can block an IP address after ten failed login attempts, but it cannot reason about whether a successful login followed the failures, whether the target host is a domain controller, or whether the source IP belongs to a trusted internal scanner. More recently, Large Language Models (LLMs) have been integrated into security workflows to assist analysts with log summarization and threat

classification. Studies have demonstrated improved triage accuracy and reduced cognitive load, but these implementations remain passive – they advise the analyst on what to do, but they do not execute any response actions [3]. Other agent-based approaches, including multi-agent systems, have achieved fast detection times, yet they either lack autonomous response capabilities, operate offline, or fail to provide transparent, auditable reasoning [4][6][7]. None of the existing solutions fully replicate the complete lifecycle of a human Tier-1 analyst: triage, investigation, decision, response, and escalation – all while maintaining an immutable record of every action [1][2][6].

The primary objective of this project is to design, implement, and validate an Agentic AI system that functions as an autonomous Tier-1 SOC analyst. The system is built upon the ReAct (Reasoning + Acting) framework. The project aims to demonstrate that such an agent can monitor a SIEM platform (specifically Wazuh) in real time, enrich alerts with threat intelligence and asset context, calculate false-positive probability and priority scores using a transparent evidence-based model, and then iteratively reason about each alert to decide whether to close it, block the offending IP, disable a compromised account, or escalate the incident to a human Tier-2 analyst. A secondary objective is to ensure full auditability by logging every thought, action, and observation to an immutable database, thereby addressing the governance and transparency gaps identified in previous work [1][2][6].

The justification for this project stems from the urgent need to reduce analyst burnout and improve incident response times without sacrificing accuracy or accountability [7]. By automating the repetitive, high-volume tasks of a Tier-1 analyst, the proposed system frees human experts to focus on complex,

high-severity incidents that require advanced judgment. The system is designed to operate alongside existing SOC infrastructure, integrating directly with the Wazuh SIEM through a Flask API and using a locally hosted GPT-OSS large language model.

The importance of this project lies in its potential to transform SOC operations. Unlike passive LLM assistants that only generate recommendations, the proposed agent actively executes containment actions – blocking IPs on both Linux and Windows, disabling user accounts, and isolating hosts – thereby closing the loop between detection and response [3][6]. Unlike rigid SOAR playbooks, the agent interprets natural language security policies in real time and adapts its investigation steps to the specific context of each alert [1]. Furthermore, unlike multi-agent systems that produce opaque decisions, the ReAct framework generates explicit, human-readable reasoning traces for every decision, providing the transparency that security teams require for trust and compliance [2][6]. The added value of this project is a complete, auditable, autonomous Tier-1 analyst that can be deployed to handle a defined set of incident types, reducing manual workload while maintaining or exceeding human speed and accuracy. Experimental results, obtained from a virtual laboratory environment simulating realistic attack scenarios, confirm that the agent autonomously detects and blocks SSH brute-force attacks across both Linux and Windows within 60 seconds – matching human speed while eliminating manual error – and correctly escalates confirmed compromises to human analysts with structured tickets and full reasoning traces.

Chapter 1: Theoretical Framework

1.1 Introduction

This chapter establishes the theoretical foundation for the proposed Agentic AI Security Supervisor. It defines Agentic AI and its core characteristics, explains the ReAct (Reasoning + Acting) framework that drives the agent's decision-making, and discusses the risks and challenges associated with deploying autonomous AI agents in cybersecurity.

1.2 Agentic AI: Concept and Characteristics

Agentic AI refers to artificial intelligence systems designed to pursue complex goals with limited direct supervision. These systems exhibit four fundamental characteristics:

1. **Autonomy:** The ability to operate independently without continuous human intervention.
2. **Perception:** The capacity to sense and interpret the environment through available data sources.
3. **Reasoning:** The capability to evaluate situations, consider alternatives, and make decisions aligned with objectives.
4. **Action:** The ability to execute decisions and influence the environment.

Kshetri (2025) [1] defines agentic AI as systems capable of "autonomously perceiving their environment, making decisions, and executing actions to achieve specified goals." He identifies five core modules that enable this capability: Interfaces (communication with users/systems), Memory (short-term context and long-term knowledge), Profile (goals and roles), Planning (developing actionable plans), and Action (executing via APIs).

Table 1 compares traditional AI agents with agentic AI.

Table 1: Comparison of AI agents and agentic AI based on Kshetri (2025) [1]

Dimension	AI Agents	Agentic AI
Definition	Task-specific, rule-based systems for routine activities	Autonomous systems pursuing complex, long-term goals with

		adaptive reasoning
Autonomy Level	Low; requires human intervention for novel situations	High; operates independently with minimal oversight
Reasoning Capability	Reactive; follows predefined logic	Proactive; uses iterative planning and learns from experience
Goal Orientation	Short-term, single-task focus	Long-term, multi-step objectives
Environmental Interaction	Limited to programmed inputs/outputs	Actively perceives and interacts with external systems (APIs, databases, sensors)
Adaptability	Static; requires reprogramming for new tasks	Dynamic; continuously learns and adapts strategies
Examples	Basic chatbots, simple automation scripts	Darktrace ENDPOINT, CrowdStrike Charlotte AI, ReliaQuest GreyMatter

1.3 The ReAct Framework: Mechanism and Operation

ReAct (Reason + Act) is a specific architectural pattern for implementing Agentic AI systems [14]. Developed to combine reasoning traces with task-specific actions, ReAct enables LLMs to generate both verbal reasoning and executable actions in an interleaved manner.

The ReAct framework operates through an iterative cycle [14]:

- **Thought:** The agent analyzes observations and considers what actions would advance the goal.
- **Action:** The agent invokes a specific tool or function.
- **Observation:** The agent receives results from the executed action.

- **Repeat:** The cycle continues until the goal is achieved or human intervention is required.

This structured approach offers several advantages for security applications [14]:

- **Transparency:** The agent's reasoning is explicitly documented, enabling audit and verification.
- **Debugability:** Errors can be traced to specific reasoning steps or tool executions.
- **Reliability:** The structured loop prevents the agent from acting without proper context.

This distinction is worth clarifying: Agentic AI is a broad concept describing a system's capability to act autonomously toward goals, while a ReAct agent is a concrete architectural pattern for implementing such systems. In essence, ReAct provides a structured and auditable blueprint for building an agentic AI system.

The ReAct framework exemplifies the principles of cognitive language agents, where the LLM operates within a structured decision cycle, managing working memory and interacting with internal and external actions as described by the CoALA framework [5].

The ReAct framework's Thought-Action-Observation cycle aligns with emerging patterns in security agent design. Jones (2025) [6] independently proposes an Observation-Reasoning-Action cycle for identity threat hunting agents, suggesting that this tripartite structure is emerging as a natural paradigm for autonomous security operations. The convergence of these independent research streams validates ReAct as a principled architecture for security applications.

1.4 Risks and Challenges of Agentic AI in Cybersecurity

While agentic AI offers transformative potential for security automation, its deployment introduces significant risks that must be carefully managed. Kshetri (2025) [1] provides a comprehensive analysis of these challenges, organizing them within a Risk Management Framework that examines threats, vulnerabilities, and potential impacts.

Expanded Attack Surface. Unlike traditional AI models confined to controlled environments, agentic AI systems interact dynamically with external tools, APIs, and data sources. Each interaction point expands the attack surface, creating new vectors for unauthorized access, data leakage, and manipulation (Kshetri, 2025) [1]. For example, compromised AI agents could be used to propagate malicious commands across interconnected systems, amplifying the damage.

Vulnerabilities in Core Components. Agentic AI often relies on Retrieval-Augmented Generation (RAG) to incorporate real-time external data. However, RAG architectures introduce their own security weaknesses, including unpatched vulnerabilities in hosting platforms, misconfigured vector databases, and susceptibility to data poisoning (Kshetri, 2025) [1]. These issues are compounded in multi-agent systems, where a breach in one agent can cascade across the entire network.

Prompt Injection and Probing. Adversaries can exploit the natural language interfaces of agentic AI through prompt injection attacks, manipulating the system into executing unintended actions. Prompt probing—crafting inputs to reveal system behavior and limitations—can also expose vulnerabilities that enable jailbreaking or unauthorized function calls (Kshetri, 2025) [1]. Such techniques are particularly dangerous when agents have access to sensitive systems or decision-making authority.

The "Rare Enemy Syndrome." Kshetri (2025) [1] introduces the concept of "rare enemy syndrome" to explain how victims can be easily deceived by novel, unfamiliar cyber threats. As agentic AI systems deploy new capabilities, both defenders and users may lack the experience to recognize emerging attack patterns, increasing the likelihood of successful exploitation.

Implications for Agentic Systems: The Risk of Over-Automation

This phenomenon is particularly relevant to agentic systems; as Stefanov et al. (2025) caution, "excessive automation carries the risk of insufficient attention to new or unexpected threats

that may fall outside the scope of pre-trained agents," [7] reinforcing the need for human oversight and adaptive mechanisms even in highly automated environments.

Governance and Oversight Challenges. The autonomy of agentic AI demands robust governance frameworks. Without continuous monitoring, explicit audit trails, and well-defined boundaries, these systems may make decisions misaligned with organizational goals or ethical guidelines. Kshetri (2025) [1] emphasizes that security in agentic AI cannot be an afterthought; it must be embedded from the outset through clear guardrails, least-privilege access, and real-time oversight.

Ghosh et al. (2025) [2] categorize agentic AI risks into three impact levels and provide detailed definitions of emergent threats, including tool misuse, memory poisoning, agent collusion, and cascading failures. Their red teaming experiments reveal that external API abuse and indirect prompt injection are highly effective in undefended systems, with safety guardrails failing in 58% of cases. These findings underscore the critical importance of embedding security controls at multiple layers – a challenge this project addresses through the ReAct framework's transparent reasoning and audit trails.

A critical challenge in deploying LLMs for security tasks is ensuring deterministic, reliable behavior. Bonfanti et al. (2026) [4] address this through careful engineering: fixing random seeds, disabling non-deterministic CUDA kernels, using greedy decoding, and enforcing explicit device placement. They note that this is 'a heuristic for true determinism' and recommend further investigation. The ReAct framework's structured reasoning loop provides additional reliability by constraining the agent's decision-making to explicit thought-action-observation cycles, reducing the space of possible behaviors and creating audit trails for verification.

The critical importance of interpretability in autonomous security agents is underscored by Jones (2025) [6], who identifies trust and interpretability as primary deployment challenges. Their agentic framework for identity threat hunting incorporates 'contextual explanations

derived from both learned patterns and symbolic rules,' but does not specify how these explanations are generated or verified. The ReAct framework addresses this gap by producing explicit, human-readable reasoning traces for every decision cycle, enabling security analysts to understand, audit, and validate agent actions.

1.5 Mitigating Agentic AI Risks with the ReAct Framework

The ReAct architecture directly addresses the “rare enemy syndrome” by generating explicit, human-readable reasoning traces for every decision cycle. When the agent encounters a novel or unexpected threat pattern, its reasoning trace documents what it observed, what it considered, and why it chose a particular action. This transparency allows human supervisors to audit the agent’s behavior, detect when the agent is operating outside its training distribution, and intervene if necessary. Unlike black-box systems that silently fail on rare threats, the ReAct agent’s structured Thought-Action-Observation loop provides an auditable record that distinguishes correct reasoning from incorrect reasoning. Furthermore, the agent is designed to escalate to a human operator when confidence is low or when no policy matches the observed situation, directly countering the risk of excessive automation [7].

The governance challenges described by Kshetri [1] are mitigated through the same explicit reasoning traces, which enable continuous monitoring and post-incident verification. By logging every thought, action, and observation to an immutable SQLite database, the system provides the audit trail required for accountability and compliance.

To address the expanded attack surface and vulnerabilities in core components, the system implements least-privilege access for all API calls, isolates tool execution in a controlled environment, and logs every external interaction. The ReAct framework’s structured loop also reduces the space of possible behaviors, making it harder for an adversary to exploit non-deterministic outputs [4].

Finally, the system incorporates a mandatory escalation mechanism for low-confidence or policy-matching failures, ensuring that a human analyst remains in the loop for novel or high-severity threats. This directly addresses the risk of over-automation highlighted by

Stefanov et al. [7] and provides the interpretability that Jones [6] identifies as essential for production deployment.

In summary, while agentic AI introduces significant risks, the ReAct framework combined with careful engineering – explicit reasoning traces, least-privilege access, immutable logging, and mandatory human escalation – provides a practical and auditable path to safe, autonomous security operations.

1.6 Conclusion

In summary, Agentic AI offers a paradigm shift from passive security tools to autonomous agents capable of perceiving, reasoning, and acting. The ReAct framework, with its explicit Thought-Action-Observation loops, provides the transparency and auditability required for security applications. However, risks such as prompt injection, expanded attack surface, and the “rare enemy syndrome” must be carefully managed – concerns that the proposed system addresses through its design.

Chapter 2: Literature Review and Research Gap

2.1 Introduction

This chapter reviews the current state of security automation, the role of LLMs in cybersecurity, and the limitations of traditional SIEM systems and SOAR platforms. It then examines recent agent-based approaches, identifies the gaps in existing work, and concludes with the research question that guides this project. The sections below systematically build the case that while individual components exist, no integrated solution provides autonomous, auditable, policy-driven Tier-1 analyst functionality.

2.2 Current State of Security Automation

Traditional security automation follows a rule-based paradigm. SOAR platforms excel at executing predefined workflows when specific conditions are met (e.g., blocking an IP after a threshold of failed login attempts). However, these systems cannot adapt to novel attack patterns that deviate from established rules, nor can they reason about complex scenarios that require understanding context, intent, or policy nuances.

Recent industry deployments demonstrate that agentic AI can overcome these constraints. For instance, ReliaQuest’s GreyMatter platform automates 98% of security alerts and reduces threat containment time to under five minutes [1]. Similarly, CrowdStrike’s Charlotte AI achieves over 98% accuracy in detection triage, automating more than 40 hours of manual work weekly [1]. These examples illustrate the paradigm shift from rule-based playbooks to adaptive, autonomous decision-making.

Recent research has demonstrated that agentic AI can not only detect threats but also generate defensive configurations automatically. Bonfanti et al. (2026) [4] present a multi-agent system that translates Cyber Threat Intelligence (CTI) reports into firewall rules using a hybrid neuro-symbolic approach. Their system extracts semantic information from CTI reports using hypernym/hyponym relationships, maps threats to MITRE ATT&CK techniques, and generates CLIPS code for expert systems to create iptables rules blocking malicious traffic. This work provides empirical evidence that AI agents can effectively automate security control configuration, achieving approximately 7% improvement in F1 score over baseline methods.

Focusing specifically on identity systems, Jones (2025) [6] demonstrates that agentic AI can autonomously monitor Active Directory and cloud identity logs to detect credential abuse, privilege escalation, and lateral movement. Their multi-agent framework achieves faster detection latency and lower false positives than rule-based systems, while also automating responses such as session termination and privilege revocation. This work validates that agentic approaches can effectively operationalize security monitoring and response in one of the most critical enterprise domains – identity and access management.

While these commercial and research systems show the potential of agentic AI, they are not yet integrated with SIEM alert triage in a way that replaces the human Tier-1 analyst. The next sections examine where LLMs and SIEMs currently stand.

2.3 LLMs in Cybersecurity

Recent research has explored integrating LLMs into security workflows. Applications include log summarization, threat intelligence analysis, and vulnerability explanation. Studies have demonstrated that LLMs can assist analysts by reducing the cognitive load associated with investigating alerts. However, these implementations remain fundamentally passive; the LLM serves as an advisor rather than an active defender. The model generates insights, but execution remains with human operators [3][8]. Olabiyi [8] specifically notes that LLMs can enhance SIEM frameworks by reducing alert fatigue, but still require human-in-the-loop for response execution.

Kshetri (2025) [1] characterizes this stage as one where LLMs serve merely as advisors, lacking the ability to act upon their understanding. Similarly, the CyberAlly system integrates GPT-4 with Wazuh to provide contextual recommendations to human analysts, but it remains a passive assistant without autonomous response capabilities [11]. Kshetri (2025) [1] notes that a fundamental distinction exists between conventional AI agents—which operate within predefined rules—and agentic AI, which exhibits adaptive reasoning, iterative planning, and the capacity to pursue long-term goals autonomously. This distinction underscores the gap that the proposed Agentic AI Security Supervisor aims to bridge. Similarly, Veluru and Manchala [9] propose using LLMs as “incident prevention copilots” in cloud infrastructure, yet their

system only recommends actions – it does not execute them. This reinforces the passive nature of current LLM security applications.

Even when LLMs are integrated directly with SIEM systems, they remain advisory, as shown in the next section.

2.4 SIEM Systems and the Triage Process

Security Information and Event Management (SIEM) systems play a pivotal role in modern Security Operations Centers (SOCs), providing a centralized platform for aggregating, analyzing, and responding to security incidents. These systems collect data from multiple sources within an organization's IT infrastructure, including network devices, servers, and applications, with the aim of detecting potential security threats [3]. Traditional SIEM systems rely heavily on rule-based correlation techniques to generate alerts. Despite their effectiveness in processing large volumes of data, they suffer from limitations such as high false positive rates and the inability to detect sophisticated attacks that do not match predefined rules [3][10]. Tellache et al. [10] emphasize that even with advanced correlation, SIEMs generate excessive false positives, overwhelming human analysts.

The triage process within SIEM systems is essential for filtering and prioritizing the alerts generated by these systems. Tier 1 SOC analysts are responsible for this initial triage, where they must quickly assess the relevance and severity of each alert. The effectiveness of this process is critical, as it impacts the SOC's ability to respond to genuine threats in a timely manner [3]. Recent work by Naraduhita et al. [12] integrates Wazuh with Jira and LLMs to automate ticket creation and response recommendations, but again stops short of autonomous containment. This highlights a recurring pattern: automation without execution.

The logical next step is to combine SIEM data with an LLM that not only analyzes but also acts. The following study attempted exactly that, yet remained passive.

2.5 Recent Study on LLM-SIEM Integration

A recent study conducted by Singh et al. (2024) [3] explored the effectiveness of integrating Large Language Models (LLMs) with Security Information and Event Management (SIEM) systems to enhance the performance of Security Operations Centers (SOCs). Table 2 summarizes their results.

Table 2: Performance metrics of LLM-SIEM integration (Singh et al., 2024) [3]

Model	Precision	Recall	F1-score
GPT-4	94%	92%	93%
GPT-3.5	90%	89%	89.5%
Llama 3	88%	85%	86.5%

The study achieved 40% faster processing and 60% reduced cognitive load, but the system remains passive – it advises but does not act. This result is crucial: even the most accurate LLM integrated with a SIEM does not close the loop from detection to response.

To move from passive advice to active supervision, we need a different architectural role for the LLM – not just a chat interface, but an executive brain over the SIEM.

2.6 Strategic Integration: LLM as a Security Supervisor over SIEM

This study exemplifies a strategic architectural shift where the SIEM acts as the “Sensory System” (collecting and aggregating data) and the Agentic LLM functions as the “Executive Brain” (interpreting context and driving decisions). Table 3 illustrates this division of labor.

Table 3: Operational phase comparison – Traditional SIEM vs. Agentic LLM

Operational Phase	Traditional SIEM Role(The Tool)	Agentic LLM Role(The Supervisor)
Event Detection	Aggregates raw logs and triggers alerts based on static, predefined patterns.[3]	Ingests the alert and interprets log telemetry using Natural Language Understanding [3]
Reasoning & Analysis	Categorizes events using rigid	Analyzes intent and context (e.g., "User'X' is

	logic(e.g., "SignatureID:1001 Matched")	accessing financial records post hours, violating Policy A").
Decision & Action	Flags the event for human review and waits for manual intervention.	Initiates a reasoning loop to decide on and execute containment actions.

This supervisor role is precisely what the emerging paradigm of Agentic AI promises to deliver. The next sections review the state of Agentic AI in security.

2.7 The Emergence of Agentic AI

Agentic AI represents a paradigm shift from passive language models to autonomous agents capable of perceiving, reasoning, and acting. In cybersecurity, early work has demonstrated the potential for agents to investigate alerts and execute response actions autonomously.

The feasibility of agentic AI in security operations is evidenced by several commercial deployments. Darktrace’s ENDPOINT uses lightweight AI agents to monitor network behavior and autonomously quarantine suspicious devices [1]. Microsoft’s Security Copilot integrates 11 task-specific agents that reduce mean time to respond by 30% and improve analyst accuracy by 35% [1].

Ghosh et al. (2025) [2] provide a comprehensive safety and security framework for agentic systems, introducing a risk taxonomy and demonstrating layered defense effectiveness (24.0% → 3.7% attack reduction).

Bonfanti et al. (2026) [4] operationalized the CoALA cognitive architecture for firewall rule generation, using hypernym/hyponym relationships to extract threat information and generating deterministic iptables rules.

Jones (2025) [6] defined an Observation-Reasoning-Action cycle for identity threat hunting, closely mirroring the ReAct framework. This independent emergence validates ReAct as a natural fit for security applications.

More recently, the CORTEX multi-agent system demonstrated that role-specialized agents can achieve higher accuracy than single-agent ReAct baselines, but at the cost of significantly higher latency and without autonomous response execution [13].

A Tier-1 SOC analyst’s daily work consists of monitoring SIEM, triaging alerts, gathering context (IP reputation, user history), following playbooks, documenting actions, and escalating when uncertain. The proposed supervisor maps directly to these tasks: monitoring via Wazuh API, triage via LLM reasoning, context via tools (`check_ip_reputation`, `get_user_history`), response via `block_ip`, `disable_user`, `isolate_host`, and escalation via `escalate_to_human`.

2.7.1 Agentic AI in Security Operations Centers

Stefanov et al. (2025) [7] proposed an autonomous multi-agent system explicitly designed to optimize SOC performance. Their architecture comprised ten specialized agents (TTD Monitoring Agent, FNR Analysis Agent, FPR Suppression Agent, TTR Response Agent, MTTR Recovery Agent, etc.). They achieved near-instantaneous detection (TTD of 0.01 seconds) and automated playbook execution. However, the authors acknowledge limitations: substantial resource requirements, lack of transparency, and risk of insufficient attention to new threats outside pre-trained agents’ scope [7].

2.7.2 Why a Single ReAct Agent Beats Multi-Agent Systems

Multi-agent systems like CORTEX [13] or Bonfanti’s framework [4] distribute work among specialized agents. While this can improve accuracy, it introduces coordination overhead, higher latency, and complexity. In the CORTEX multi-agent system, the median end-to-end latency per alert was 152.4 seconds, compared to 44.6 seconds for a single-agent ReAct baseline – a 3.4× slowdown [13]. Moreover, CORTEX only escalates alerts to human analysts; it does not execute any containment actions such as blocking IPs or disabling accounts. A single ReAct agent uses a simple Thought → Action → Observation loop, achieving comparable accuracy while executing responses – something multi-agent systems in the

literature do not do. Furthermore, the ReAct loop natively produces an audit trail: every thought, action, and observation is logged.

Despite these advances, no existing system – commercial or research – provides all three capabilities needed to replace a human Tier-1 analyst. The next section synthesizes this gap explicitly.

2.8 Synthesizing the Gap: What Existing Systems Miss

In real Security Operations Centers, the challenge is not just about detection – it is about volume, speed, and trust. SOC analysts face up to tens of thousands of alerts per day, with false-positive rates approaching 99% [3]. Studies show that 30–50% of an analyst’s time is spent on repetitive triage, leading to alert fatigue, burnout, and delayed response [7]. While the LLM-SIEM integration studied by Singh et al. (2024) improves classification, it remains passive – it tells the analyst what to do but does not act.

Table 4 contrasts the current state-of-the-art with the proposed advancement.

Table 4: Current state vs. proposed contribution

Domain	Current State	The Gap	Contribution of This Project
LLM Capability	LLMs are used passively for classification and analysis. They advise but do not act.	Lack of autonomous action.	Transforms the LLM into a proactive agent that can execute security actions
Decision Output	The system states: "This alert is critical."	No autonomous response.	The system states: "This alert is critical" and then takes action, such as blocking an IP.
Transparency	The system acts as a "Black Box," with limited insight into its decision-making process.	Lack of auditability and trust, which are critical in security.	Employs the ReAct framework to generate explicit reasoning traces, creating a "Glass Box"

			where every step is documented.
Action Execution	Requires a human analyst to investigate and take action.	Delayed response times and increased analyst workload	Automates routine containment actions(e.g., blocking IPs,disabling accounts),escalating only complex cases.

The critical gap lies in the absence of systems that combine:

- ❖ The contextual reasoning and natural language understanding of LLMs
- ❖ The ability to execute security actions autonomously
- ❖ Structured decision-making frameworks that ensure reliable, auditable behavior

Kshetri (2025) [1] corroborates this assessment, observing that ‘a gap continues to exist between LLM assistants and fully autonomous agents.’ He further emphasizes that closing this gap requires not only technical capability but also robust governance, transparency, and risk management. His proposed Risk Management Framework—examining threats (e.g., expanded attack surface, prompt injection), vulnerabilities (both objective and subjective), and potential impacts—provides a structured lens for designing safe agentic systems. This framework underscores the importance of audit trails and explicit reasoning, which are central to the ReAct architecture adopted in this project.

The need for structured, auditable decision-making in autonomous security agents is further highlighted by Ghosh et al. (2025) [2], who demonstrate through extensive red teaming that undefended agentic systems exhibit attack success rates of 24.0%, which can be reduced to 3.7% through layered defenses. However, their framework does not prescribe a specific reasoning architecture; this project addresses that gap by adopting the ReAct framework, which provides explicit reasoning traces for transparency and verification.

Existing solutions offer either rigid automation without reasoning (SOAR) or reasoning without action capabilities (passive LLM assistants). Furthermore, frameworks for ensuring that autonomous agents operate safely within security contexts—without becoming vectors for new vulnerabilities—remain underdeveloped. This project addresses this gap by developing an Agentic AI Security Supervisor that reasons about security events and acts upon its conclusions within a controlled, auditable framework.

The need for practical, verifiable agent implementations is underscored by Bonfanti et al. (2026) [4], who note that 'existing AI-based approaches struggle with the inherent complexity of automatically categorizing sensitive data, which is often exacerbated by severe data imbalance across different threats.' Their work addresses this by using semantic relations to improve extraction performance, achieving superior results on imbalanced cybersecurity datasets. However, their system remains research-focused and generates rules for an expert system rather than executing actions directly. This project extends such work by implementing a ReAct agent that not only understands security events but also executes responses autonomously while maintaining auditability through explicit reasoning traces.

Jones (2025) [6] notes that while agentic AI systems show promise for identity threat detection, they face challenges including 'computational overhead for real-time analysis, the need for high-quality labeled and contextual data, and integration with existing security infrastructures.' Crucially, they identify trust and interpretability as critical requirements, noting that 'security teams must understand the rationale behind autonomous agent decisions.' The proposed ReAct-based supervisor directly addresses this gap by generating explicit reasoning traces that document every decision, providing the interpretability that production deployments demand.

Furthermore, frameworks for ensuring that autonomous agents operate safely within security contexts—without becoming vectors for new vulnerabilities—remain underdeveloped. This challenge is evident in recent agentic systems such as that proposed by Stefanov et al. (2025), who acknowledge that their architecture, while effective for predefined metrics, carries the risk of "insufficient attention to new or unexpected threats that may fall outside the scope of pre-trained agents." [7] Their approach requires substantial resources and periodic retraining,

limiting adaptability to rapidly evolving threat landscapes. These limitations underscore the need for architectures that combine predictive power with explicit reasoning, policy interpretation, and transparent decision-making.

2.8.1 The Gap in Autonomous Response (CORTEX & Others)

Recent attempts to move beyond passive assistance have emerged. The CORTEX multi-agent system, for example, achieved a triage F1 score of 0.78 with a false-positive rate of 14% – a significant improvement over rule-based SIEM [13]. However, CORTEX only escalates to a human; it does not execute any containment actions (e.g., blocking an IP or disabling a user). Moreover, its latency of 152 seconds per alert is too high for real-time response [13]. Other multi-agent frameworks (e.g., Bonfanti et al., 2026) generate firewall rules but offline, without real-time interaction or human escalation. Stefanov et al. (2025) [7] achieve fast detection (0.01 sec) but lack transparency and do not interpret natural language policies.

The gap is therefore threefold:

1. No system that both reasons about security events and executes responses autonomously.
2. No system that provides explicit, auditable reasoning traces for every decision.
3. No system that interprets natural language security policies in real time and maps them to actions.

The proposed single ReAct agent directly addresses all three. Having established this threefold gap, the following research question guides the design of a system that fills it.

In direct comparison, CORTEX [13] achieves moderate accuracy gains over a single ReAct agent but fails to provide autonomous containment and incurs prohibitive latency, whereas the proposed agent executes cross-platform blocking within 60 seconds while maintaining full auditability.

2.9 Distinction from Existing Security Technologies

2.10 Research Question

How can an Agentic AI system, utilizing a ReAct framework, be designed and implemented to autonomously and effectively monitor system health, audit access logs for compliance violations, and initiate corrective actions against security breaches, thereby reducing the need for human intervention in Security Operations Centers?

2.11 Conclusion

The literature confirms that while SOAR platforms and passive LLM assistants have improved certain aspects of security operations, no existing system fully replaces the Tier-1 SOC analyst's complete workflow – triage, investigation, decision, response, and escalation – with auditable reasoning. The research gap is threefold: autonomous response, explicit reasoning traces, and real-time policy interpretation. This project directly addresses these gaps through a single ReAct agent integrated with Wazuh SIEM.

Chapter 3: Proposed Solution

3.1 Introduction

This chapter presents the detailed architectural design of the Autonomous SOC Tier-1 Analyst Agent. It describes the alert ingestion and normalization layer, the four-stage autonomous triage engine, the ReAct agent core with its perception tools, the correlation engine, and the incident report and audit trail mechanism.

3.2 Proposed Solution

The system is engineered to bridge the gap between passive LLM-based security assistants and rigid, rule-based SOAR platforms. It integrates directly with the Wazuh SIEM, functioning as an intelligent, proactive supervisor that executes the complete triage, investigation, decision-making, and response lifecycle of a human Tier-1 analyst. The architecture instantiates the ReAct framework into a practical, modular, and fully auditable security operations pipeline, with every reasoning step and action permanently logged.

At the core lies the Security Agent (ReAct Engine) built using LangChain and LangGraph. The agent receives alerts from Wazuh (via the Wazuh Manager) and enters an iterative reasoning loop consisting of Thought → Action → Observation. It leverages a large language model (GPT-OSS served by Ollama) for decision-making, and invokes a set of software tools (information gathering, response actions, escalation, and policy retrieval) to conduct investigations and execute real-time countermeasures – such as blocking IP addresses (`block_ip`) or disabling user accounts (`disable_user`).

All decisions and actions are stored in a State Manager (SQLite) which maintains session memory, audit logs, and an immutable decision trace (ReAct traces), ensuring full transparency. The agent also consults external threat intelligence (e.g., VirusTotal) and a policy store containing NIST, MITRE ATT&CK, and internal security rules to guide its reasoning process.

By integrating these components, the proposed system transforms Wazuh from a passive detection platform into an autonomous security supervisor that not only understands alerts but

also makes evidence-based decisions and executes responses automatically – while keeping the human analyst in the loop for escalation when necessary.

3.3 System Architecture Overview

The system's design philosophy prioritises a layered, evidence-based decision process. As shown in the high-level architecture diagram (Figure 1), data flows sequentially from raw SIEM ingestion through a multi-stage triage engine, culminating either in autonomous response by a ReAct agent or a structured escalation to a human analyst. The pipeline is owned by a central SOCAgent class and is exposed and managed through a Flask API layer that provides both real-time webhook reception and periodic polling capabilities.

All persistent state is stored in two SQLite databases: an audit database (audit.db) for alerts, triage decisions, investigation logs, incident reports, and correlation chains, and a checkpoints database (checkpoints.db) used by the LangGraph agent to persist reasoning state across cycles.

3.4 Alert Ingestion and Normalization Layer

Security events are collected from the Wazuh platform via two complementary mechanisms:

- **Real-time webhook:** A Flask endpoint (POST /webhook) receives JSON alerts pushed by Wazuh's custom-webhook integration module, configured to forward any rule with a level ≥ 5 . This provides near-instantaneous delivery for critical alerts.
- **Background poller:** A dedicated thread polls the Wazuh Indexer (Elasticsearch/OpenSearch) REST API every 30 seconds, retrieving alerts timestamped since the last checkpoint. This dual-channel design ensures no alert is lost even if the webhook is temporarily unavailable.

Raw Wazuh alerts are immediately processed by the `normalize_wazuh_alert` function, which converts the complex, often nested JSON into a clean, predictable `NormalizedAlert` object. The normaliser performs several critical functions:

- Filters out low-severity noise: any alert with `level < 3` is discarded. Furthermore, Windows events with an unresolvable source IP (0.0.0.0) and a level below 7 are suppressed to avoid local logon noise.
- Maps the Wazuh numeric severity level (0–15) to a human-readable label: `level ≥ 13` → CRITICAL, `level ≥ 10` → HIGH, `level ≥ 7` → MEDIUM, otherwise LOW.
- Extracts the source IP with fallback logic that checks data fields (`srcip`) and Windows event-specific paths (`eventdata.ipAddress`).
- Extracts the target user and host, and captures any MITRE ATT&CK technique and tactic already annotated by Wazuh rules.
- Generates a deduplication key by hashing the timestamp prefix, rule ID, source IP, and agent name. Alerts that produce an identical key within a short timeframe are treated as duplicates and not re-processed, drastically reducing downstream work.

Alert Aggregation: The `AlertAggregator` groups alerts arriving within a 5-second window for the same source IP. If an alert has `severity ≥ 10`, indicates a successful action (keyword “success”), or is a privilege escalation (rule 5402/5403), the window is bypassed and the alert is released immediately. Otherwise, the highest-severity alert in the bucket is released after the window expires. This prevents a flood of identical failed login alerts from triggering hundreds of identical investigations.

3.5 Autonomous Triage Engine

The core intelligence of the system resides in a sequential, four-stage Triage Engine. Each stage replicates a cognitive function that a Tier-1 analyst performs manually within the first

two minutes of receiving an alert, moving from broad context gathering to a definitive preliminary verdict.

Justification of thresholds: The numerical thresholds used in false-positive probability, priority scoring, and escalation decisions (e.g., +0.70 for whitelist, +25 for compromise, ≥ 40 for CRITICAL) were determined empirically through iterative testing on over 50 simulated alerts covering a range of attack scenarios. The values were tuned to balance sensitivity (avoiding false negatives) and specificity (minimising false positives) while aligning with common SOC operational practices.

Stage 1 – Context Enrichment

The ContextEnricher module queries multiple internal and external sources to build an AlertContext object for the alert. It does the following:

- Looks up the asset inventory in the audit database to determine the target host's criticality (CRITICAL, HIGH, MEDIUM, LOW) and whether it is a domain controller.
- Queries the alert history for the same source IP over the preceding 24 hours to compute total alerts, distinct triggered rules, and first/last seen timestamps.
- Checks if the source IP is listed in the internal whitelist (e.g., a scheduled internal vulnerability scanner).
- Identifies recurring patterns by checking whether the same alert type from the same source has occurred more than five times in the same timeframe over the past week.
- Retrieves the behavioural history of the targeted user account.

- Consults external threat intelligence: the VirusTotal API for the source IP's malicious count and the AbuseIPDB API for its abuse confidence score.
- Determines, by scanning subsequent alerts, whether any successful authentication event (Wazuh rules 5501, 5502) occurred from the same IP immediately after the failures – a critical indicator of compromise.

Stage 2 – False Positive Assessment

The FalsePositiveDetector calculates a probability score (0.0 to 1.0) that the alert is a false positive, starting from a neutral score of 0.5. The score is adjusted using a set of weighted evidence indicators:

- Indicators that increase FP probability (score increases):
- Source IP is in the whitelist: +0.70
- Identical alert recurs at the same time daily (scheduled scan): +0.35
- Source IP falls within internal/private ranges: +0.25
- No threat intelligence hits (VirusTotal clean): +0.15
- Indicators that decrease FP probability (score decreases, pointing toward true positive):
- VirusTotal malicious count > 10: -0.50
- AbuseIPDB confidence score > 80%: -0.40
- A successful login followed the failed attempts: -0.80
- The target user is a privileged account (root, admin): -0.30
- Activity occurred outside business hours: -0.15

The final score is clamped between 0.0 and 1.0. Alerts scoring > 0.70 receive a verdict of FALSE_POSITIVE and are typically auto-closed. Alerts scoring < 0.30 are considered TRUE_POSITIVE. Scores between 0.30 and 0.70 return an UNCERTAIN verdict and are sent to the ReAct agent for deeper investigation.

Stage 3 – Priority Scoring

For alerts not classified as obvious false positives, the PriorityScorer assigns a severity level using a transparent, additive weighting formula that combines the Wazuh raw signal with organisational context and threat intelligence:

- Base severity: The Wazuh rule level (0-15) is added directly.
- Asset criticality weight: CRITICAL assets contribute 15 points, HIGH 10, MEDIUM 5, LOW 2.
- Threat intelligence confirmation: If VirusTotal malicious count > 20, +15 points; >10, +10 points; or if AbuseIPDB score > 80, +8 points.
- Compromise indicators: A successful authentication after failures adds +25 points, instantly elevating the alert toward CRITICAL.
- Privileged target: If the target username is “root”, “admin”, or “administrator”, +8 points.
- Temporal factors: Activity outside business hours adds +5 points.
- Attack volume: More than 100 similar alerts in the last hour adds +10 points; >50 adds +5 points.

The cumulative score is then mapped to a final priority: CRITICAL (≥ 40), HIGH (≥ 25), MEDIUM (≥ 15), LOW (< 15). The detailed score breakdown is saved to the audit trail, making the decision fully interpretable.

Stage 4 – Escalation Pre-Decision

The EscalationDecider makes a critical governance decision: can the agent handle this incident alone, or must a human be involved? It first evaluates a set of mandatory escalation triggers – any single one forces an immediate ESCALATE decision with an IMMEDIATE urgency tag:

- A successful login followed the brute-force attempts (compromise confirmed).
- The attack has spread to multiple hosts (lateral movement).
- The target is a domain controller or backup server.
- Signs of data exfiltration or ransomware behavioural patterns are present.

- The required response (e.g., full network isolation) exceeds the agent’s autonomous privileges.

If none of the mandatory triggers are activated, the module then evaluates the priority score: all CRITICAL-level alerts and any HIGH-level alert with an UNCERTAIN FP verdict are escalated with HIGH or MEDIUM urgency, respectively. Only incidents that pass all gates – low-to-medium severity, high-confidence true positive, no mandatory flags – are cleared for autonomous handling via the ReAct agent.

3.6 The ReAct Agent Core

When the Escalation Decider returns `HANDLE_AUTONOMOUSLY`, the system constructs a comprehensive text prompt containing the full alert details, the triage pre-assessment results, the enriched context, and the list of available tools. This prompt is forwarded to the ReAct agent, an LLM-driven autonomous investigator built with LangChain and LangGraph.

The agent’s behaviour is governed by a detailed system prompt (`ANALYST_SYSTEM_PROMPT`) that instills a structured, six-step investigative methodology:

`UNDERSTAND → ENRICH → CORRELATE → ASSESS → DECIDE → REPORT`

The prompt further encodes critical policy constraints, such as “NEVER block internal IPs without escalating first” and “ALWAYS escalate if a successful login followed failed attempts,” as well as block duration rules (`LOW = 30s`, `MEDIUM = 60s`, `HIGH = 300s`, `CRITICAL = 3600s`).

The agent operates in a strict Thought → Action → Observation loop. In each cycle, it reasons about what it knows, selects exactly one tool to invoke, and stops. It processes the tool’s observation before generating the next thought. This constrained interaction prevents runaway behaviour and ensures that every external action produces an auditable log entry.

3.7 Perception Layer (Tools)

The agent is equipped with a set of software connectors (tools) that enable it to interact with the security environment. Table 5 details the tools and their corresponding analyst tasks.

Table 5: Agent tools and corresponding analyst task

Tool Name	Parameters	Analyst
query_wazuh_logs	query (string), time_range (string)	Search the Wazuh indexer for raw logs related to a specific IP, user, or event ID.
get_user_history	username (string)	Retrieve the recent activity profile of a user account (logins, commands, failures).
check_ip_reputation	ip (string)	Query VirusTotal for the IP's detection ratio by antivirus engines.
block_ip	ip (string), duration (int)	Execute a cross-platform firewall rule (iptables on Ubuntu, netsh via SSH on Windows) to contain the threat.
disable_user	username (string)	Disable a compromised user account via the Wazuh API.
isolate_host	hostname (string)	Quarantine an entire host from the network by adjusting firewall rules.
escalate_to_human	message (string)	Create a structured, high-priority incident ticket in the audit database

		for human review.
get_policy	policy_name (string)	Retrieve the organisational security policy text for a given scenario (e.g., SSH brute force).
check_known_threat	source_ip, rule_id	Determine if the current alert matches a known, confirmed threat signature to accelerate triage.
query_recent_alerts	minutes, source_ip	Gather aggregated alerts from the audit database to identify repeating or correlated threats.

After completing its investigation, the agent does not simply write free text; it is forced to output a rigorously formatted Final Answer containing the fields VERDICT, CONFIDENCE, PRIORITY, ACTION, ESCALATED, MITRE, and SUMMARY. This structured output ensures machine-readable consistency and facilitates automated KPI computation.

3.8 Correlation Engine

After the agent delivers its verdict, a dedicated CorrelationEngine analyses the processed alert in the context of other events within a configurable 30-minute correlation window. The engine is pre-loaded with definitions of known multi-stage attack chains, such as:

- SSH Brute Force to Compromise: A sequence of authentication failures (rule 5760/5763) immediately followed by a successful login (5501/5502).
- Lateral Movement: A successful brute force on one host followed by a new brute force pattern against a different target from the same source.
- Privilege Escalation: A series of failed sudo commands followed by a successful one.

If a complete or partial chain is detected, the attack timeline is reconstructed and attached to the incident report, providing the human analyst with a chronological narrative of the intrusion.

3.9 Incident Report and Audit Trail

The final stage of the pipeline is the ReportGenerator, which compiles every piece of gathered intelligence into a structured incident report. This report includes an executive summary, full technical details, the complete priority score breakdown, threat intelligence results, the correlated attack chain (if any), the complete ReAct agent reasoning trace (every Thought, Action, and Observation), a list of actions taken, and a clear escalating/closure recommendation.

Crucially, every step of the entire pipeline – from the original raw alert through the triage engine’s intermediate scores to every tool invocation by the agent – is stored immutably in the SQLite audit database. This provides the “glass box” auditability necessary for trust and post-incident review, directly addressing the governance and interpretability challenges described in the literature.

3.10 Reasoning Layer (LLM Core)

A Large Language Model serves as the decision-making engine. It interprets natural language security policies, analyzes observations from the perception layer, and determines appropriate actions based on its understanding of the situation and defined policies. The LLM is configured with a low temperature (0.1) to ensure deterministic, reliable, and policy-adherent responses – a critical requirement for security operations.

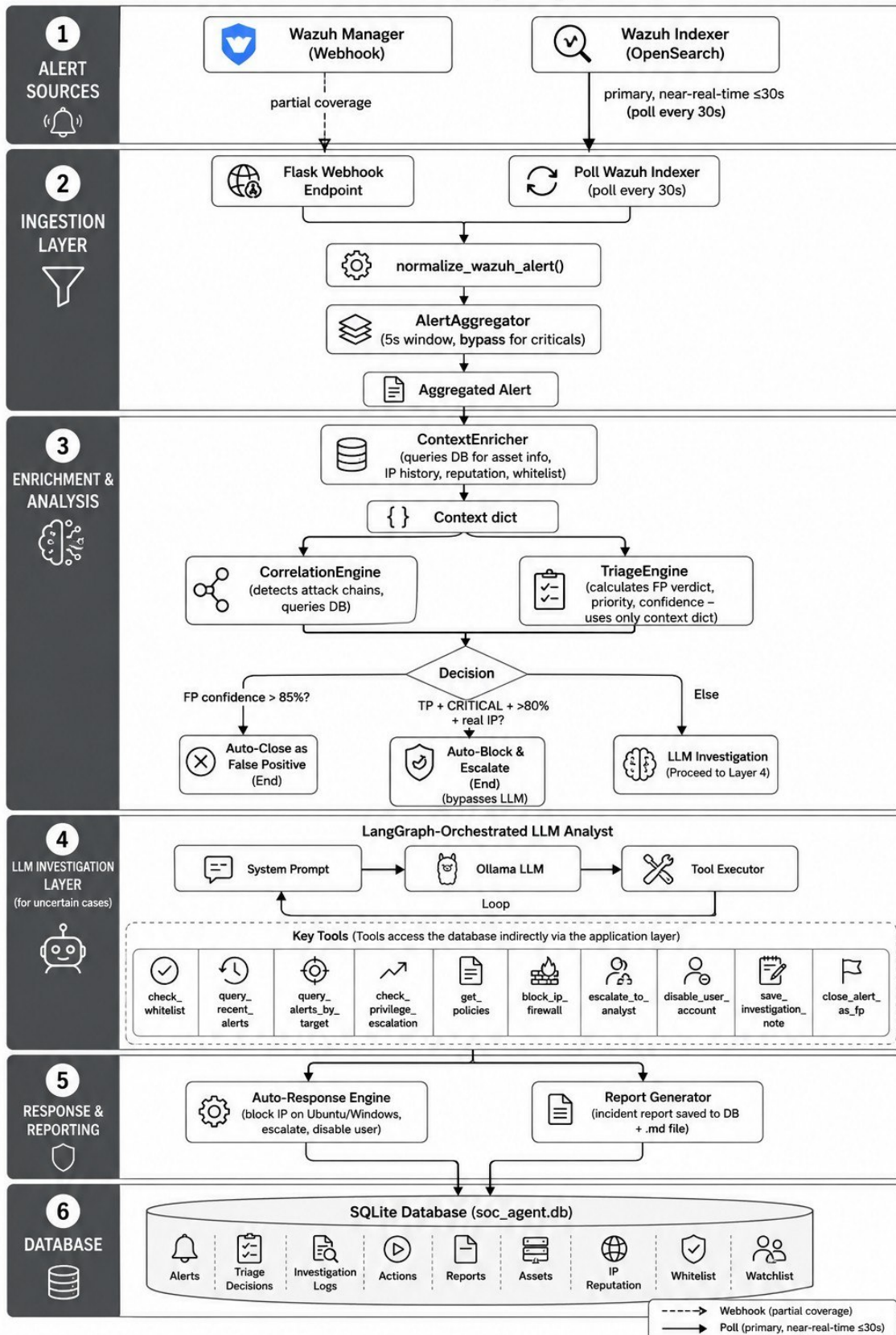
3.11 Scope of Supervision

To maintain focus and feasibility, the initial implementation monitors:

- Access Logs: Failed authentication attempts, privilege escalation events, and anomalous access patterns.
- Compliance Violations: Activities that contravene predefined security policies, such as multiple failed logins from a single source or access attempts outside authorised hours.

Figure 1: High-Level System Architecture of the Agentic AI Security Supervisor

Autonomous SOC Tier-1 Analyst Agent – Final Architecture



Chapter 4: Practical Application and Results

4.1 Introduction

This chapter details the implementation of the proposed system in a virtual laboratory environment (Ubuntu, Windows, Kali) and presents the experimental results from two attack scenarios. It covers the Python libraries, the Wazuh SIEM integration, the GPT-OSS model via Ollama, and the evaluation of the agent's performance.

4.2 Challenges of Integrating AI with Cybersecurity

Integrating artificial intelligence with cybersecurity tools presents significant challenges that must be addressed to build robust and secure systems. The primary challenge lies in ensuring that AI-driven systems do not compromise the very security they are designed to protect. This requires careful architectural decisions, thorough testing, and continuous monitoring of AI behavior within security environments. Additionally, the dynamic nature of cyber threats means that AI models must be regularly updated and retrained to maintain their effectiveness against evolving attack patterns.

4.3 Python in Cybersecurity

Python has proven to be a fundamental language in our project, enabling the development of an autonomous Agentic AI Security Supervisor that integrates with Wazuh SIEM and executes real-time response actions. The following aspects highlight how Python was applied:

Automation and Incident Response: Python scripts were developed to automate the entire investigation workflow. Using requests, the agent communicates with Wazuh REST API to obtain authentication tokens, query alerts via the Indexer (Elasticsearch), and trigger Active Response commands (e.g., firewall-drop and disable-account). A Flask-based webhook endpoint runs in a separate thread to receive alerts from Wazuh in real time, ensuring non-blocking processing. A threading Timer is used to automatically remove firewall rules after a configurable period, preventing permanent blocking.

LLM-Powered Reasoning with ReAct Framework: The agent leverages `langchain_ollama.ChatOllama` to connect to a locally hosted LLM (GPT-OSS via Ollama). The LangGraph library builds a state machine that implements the ReAct (Reason + Act) pattern. The agent iterates through cycles of Thought → Action → Observation, calling Python-defined tools such as `get_policy`, `query_wazuh_logs`, and `block_ip`. The LLM is bound to these tools using `bind_tools`, enabling it to decide which tool to invoke based on the alert context.

Cross-Platform Response Actions: Python's `subprocess` module executes `iptables` commands on Ubuntu to block malicious IP addresses at the network level. For Windows targets, the `paramiko` library establishes an SSH connection to the Windows VM, where `netsh advfirewall`

commands are sent to block IPs. These actions are integrated into the `block_ip` tool, providing a unified blocking mechanism across heterogeneous environments.

Policy Management and Knowledge Base: Security policies are stored in a `policies.json` file and loaded using Python's `json` module. The `get_policy` tool retrieves the appropriate policy (e.g., for brute-force attacks, privilege escalation, or port scanning) and guides the agent's investigation steps. This eliminates hard-coded rules and allows flexible policy updates without modifying the core code.

4.4 Virtual Environment Setup using VirtualBox

In implementing the project, we relied on building a virtual laboratory environment that simulates a small-scale real network containing the Wazuh platform and its surrounding components (agents on target systems and an attacking machine). To achieve this, VirtualBox was used to create multiple virtual machines (VMs) and run them on the same physical computer. VirtualBox allows emulating devices with different operating systems and enabling communication between them according to configurable network settings. This provided us with a closed, secure testing environment to conduct experiments without affecting a real network or exposing us to external risks

4.4.1 Internal Network Communication (NAT & Host-Only):

NAT Adapter:

This adapter provided the virtual machines (Wazuh server, target systems, and attacker machine) with internet access through the host machine. This setup was necessary for downloading security updates, installing packages and tools such as `hydra` and `curl`, and pulling container images when needed.

Host-Only Adapter:

This adapter was dedicated to creating an isolated internal network with no internet access, used exclusively for direct communication between the virtual machines. Static IP addresses were assigned within this range (e.g., `192.168.56.x`). This network was relied upon to enable the following components to communicate reliably:

Sending data from the Wazuh agent on Windows to the Wazuh manager, Delivering alerts from the Wazuh manager to the intelligent agent via the `/webhook` endpoint,

Directing attacks from the Kali Linux machine toward the target systems (e.g., brute-force attacks).

This design combines two benefits: providing internet connectivity for maintenance when needed (via NAT), and fully isolating the sensitive internal network from the outside world to ensure experiment safety (via Host-Only). This setup mirrors a real-world scenario where critical servers are placed in an internal network or DMZ that is not directly accessible from the internet.

4.4.2 Virtual Environment Components:

1: Ubuntu VM

This machine hosts three key components:

- ❖ **Wazuh Manager:** the central SIEM server that collects logs from all agents, correlates events, and generates alerts.
- ❖ **Wazuh Agent:** monitors the Ubuntu system itself and sends its logs to the Manager.
- ❖ **Agentic AI (the intelligent supervisor):** includes the ReAct framework (Reason + Act) for step-by-step reasoning, and Python Code implementation security tools (`block_ip`, `disable_user`, `query_wazuh_logs`) that communicate with Wazuh API and Ollama.

2: Windows VM

A typical endpoint, running only the Wazuh Agent. It gathers Windows system logs (Event Logs, login attempts, privilege changes) and forwards them to the Wazuh Manager.

3: Kali VM

Dedicated attacker machine running Kali Linux. It is used to execute controlled attacks (e.g., password brute-force with hydra) to test the platform's ability to detect and automatically respond (e.g., IP blocking, account disabling).

Table 6: Description of environmental components

VM	Role	Operating System	Host-Only IP Address
	• Wazuh Manager (central SIEM) – receives logs and generates alerts.		

Ubuntu VM	• Wazuh Agent – monitors Ubuntu itself. • Agentic AI (ReAct + Code) – the intelligent supervisor.	Ubuntu 22.04 (64-bit)	192.168.56.103
Windows VM	Endpoint with Wazuh Agent only – sends Windows Event Logs to Wazuh Manager.	Windows 10 Pro (64-bit)	192.168.56.104
Kali VM	Attacker machine – used to execute test attacks (e.g., SSH(brute force) to evaluate detection and automated response.	Kali Linux (64-bit)	192.168.56.105

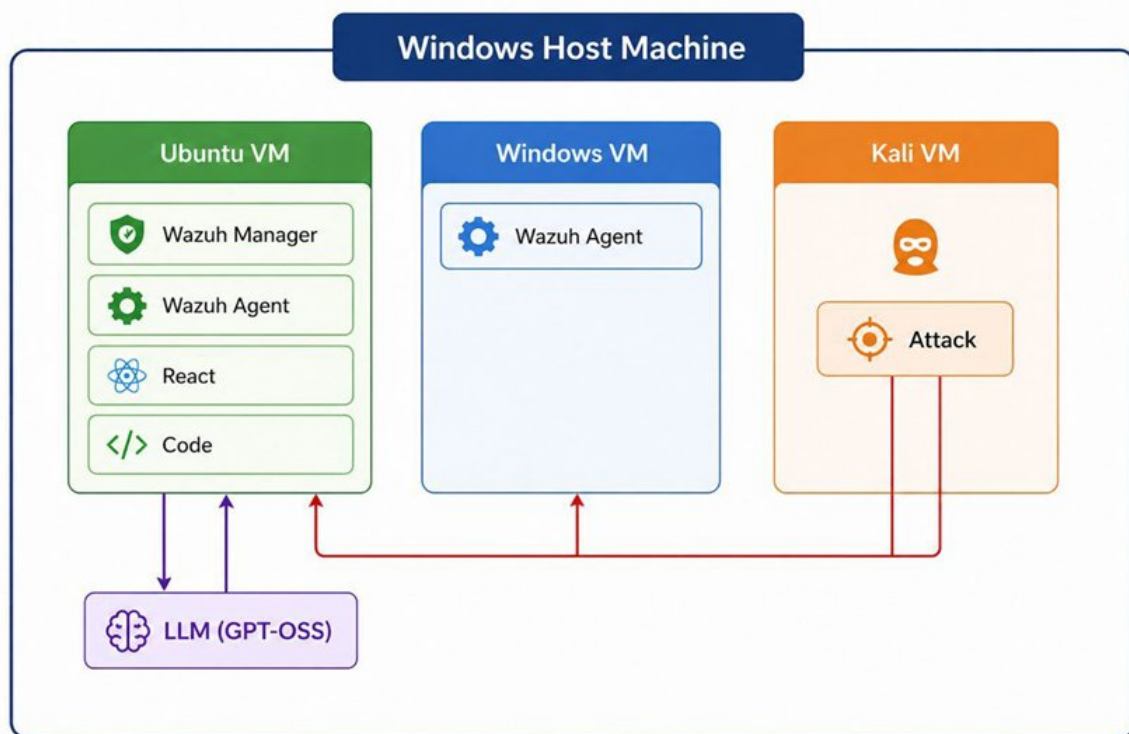


Figure 2:Description of environmental components

4.5 Libraries and Artificial Intelligence Technologies Used

- **LangChain and LangGraph:** LangChain helps the intelligent agent communicate with large language models (e.g., GPT-OSS) and management tools. LangGraph

builds the ReAct reasoning loop (Thought → Action → Observation) and controls the agent's workflow step by step.

- **langchain_ollama.ChatOllama:** This module connects the agent to the GPT-OSS model running via Ollama. It allows the model to read security policies and invoke tools such as `block_ip` and `disable_user`
- **requests library and subprocess module:** requests is used to communicate with Wazuh APIs (fetching tokens, sending block commands, querying alerts). Subprocess executes iptables commands directly on Ubuntu from within the Python code.
- **Paramiko:** A library that enables the agent to establish an SSH connection to the Windows VM, so it can remotely execute netsh advfirewall commands to block IP addresses there.
- **sqlite3:** Used for checkpointing together with LangGraph. It preserves the investigation state, allowing the agent to remember steps and resume work without losing context.
- **Flask integrated server (webhook endpoint):** A lightweight built-in web server running on port 5001. It receives alerts from the Wazuh Manager in real time (via POST) and forwards them to the agent for analysis previous and automated response.

4.6 Wazuh platform as an open-source SIEM system

Wazuh is an open-source Security Information and Event Management (SIEM) system. It consists of three main components: Wazuh Manager (the central server that receives logs from agents, analyzes them, and generates alerts), Wazuh Indexer (an Elasticsearch database that stores and indexes alerts), and Wazuh Dashboard (a Kibana-based graphical interface for viewing alerts, managing agents, and monitoring statistics). Wazuh was chosen for this project for several essential reasons

Open-source and free: Unlike some expensive commercial platforms, Wazuh is released under an open-source license, making it suitable for academic and research projects and allowing free modification and customization.

Focused on Scalability: Wazuh features a distinctive architecture that allows it to scale to handle large volumes of data. It can be deployed across a cluster of servers and dozens or even hundreds of agents with minimal issues. Studies indicate that Wazuh has gained wide recognition for its flexibility and scalability, in addition to its robust capabilities even in a compact version. These advantages make it a suitable choice for the project, which may require managing a large volume of security logs, especially when simulating diverse and frequent attacks.

Active Response capabilities: Wazuh offers an Active Response feature that allows the Manager to execute immediate automated actions, such as blocking attacking IP addresses (using the `firewall-drop` command) or disabling compromised user accounts (using the `disable-account` command). This feature is the backbone of the blocking and disabling mechanisms relied upon by the intelligent agent in our project.

4.7 Large Language Model GPT-OSS and Running it via Ollama

The GPT-OSS large language model is an open-source model developed by OpenAI, specifically designed for agentic tasks that require tool use and multi-step execution. We chose this model (the 20b-cloud version) to serve as the analytical and generative engine of the platform. With its large size (20 billion parameters), the model is capable of understanding complex contextual text inputs and producing coherent, logical natural language outputs. It also effectively supports function calling. The reasons for selecting GPT-OSS are as follows:

High performance in agentic tasks: GPT-OSS models have demonstrated outstanding reasoning abilities and the capacity to follow complex instructions, especially in tasks that involve calling external tools (such as `block_ip`, `disable_user`, `query_wazuh_logs`) and synthesising results. We need a model that can break down a complex security event (which may involve response steps from a knowledge base) and produce an appropriate recommendation – a capability that GPT-OSS provides thanks to its training on diverse agentic scenarios.

Local (or private-cloud) operation without data leakage: Although we used the cloud version (20b-cloud) of the model via Ollama, the service provides data isolation and does not permanently store inputs. For highly sensitive data, the model can be fully operated locally using the same Ollama tool. We were reluctant to send descriptions of security incidents to general-purpose cloud models such as GPT-4 for privacy and confidentiality reasons. Local or semi-local (cloud with privacy commitments) models offer a suitable compromise.

Native tool use and structured outputs: GPT-OSS excels at recognising when a tool should be called, extracting the correct parameters, and handling the results returned by tools. This capability is at the heart of our project, where the intelligent agent relies on calling functions such as `block_ip`, `disable_user`, and `query_wazuh_logs` based on the model's decisions. The model also supports generating structured outputs (e.g., JSON), which facilitates automated analysis of decisions.

Active community and continuous updates: GPT-OSS is one of the newest open-source models from OpenAI and has received significant attention from the developer community. It is available through Ollama in multiple flavours (local and cloud) and is continuously updated. This ensures that if we encounter any issue or need to fine-tune the model, it will be feasible and well-documented within the research community.

After installation, we asked the model about its capabilities in analyzing security logs to ensure that the model was working correctly as described:

4.8 System Overview and Implementation

This chapter presents the experimental results obtained from deploying and testing the Agentic AI Security Supervisor in the virtual laboratory environment described in Chapter 4. The system was evaluated against controlled attack scenarios executed from the Kali VM targeting the Ubuntu VM and the Windows VM. The results demonstrate that the ReAct agent successfully detects, investigates, and responds to security events autonomously, producing complete reasoning traces and structured incident escalations without human intervention.

4.9 Scenario 1: SSH Brute-Force Attack — Autonomous Containment

In this scenario, the Kali VM (192.168.56.105) launched a dictionary-based SSH brute-force attack against the Ubuntu VM using the Hydra tool. Wazuh Rule 5763 fired after detecting more than ten failed login attempts from the same source IP within a five-minute window, and the generated alert was forwarded to the agent via the Flask webhook. Figure 3 presents the agent's complete ReAct reasoning trace for this scenario.

The agent first retrieved the applicable policy (Standard SSH Brute Force) using `get_policy`, identifying the investigation requirements and the required response action. It then called `query_wazuh_logs` and found 669 failed authentication events from 192.168.56.105 within five minutes, all targeting the root account. No successful logins were detected. Based on this evidence, the agent classified the incident as a contained brute-force attempt and executed `block_ip` on both the Ubuntu VM (via iptables) and the Windows VM (via SSH-delivered netsh commands), with a 60-second block duration as specified by the policy. No escalation was triggered, as the policy's escalation condition — a successful login following the failures — was not satisfied.

This result of the agent in the Brutforce attack:

```

🚩 Agent's reasoning & actions:
**Conclusion – SSH Brute-Force Alert (192.168.56.105)**

1. **Policy** – Retrieved **Standard SSH Brute Force** policy.
  *Trigger:* ≥10 failures from a single IP in 5 min.
  *Investigation:* count failures, IP trust level, target user, look for past success.
  *Response:* block *external* source IP for 60 sec.

2. **Investigation**
  • Queried Wazuh for the last 5 min of events from 192.168.56.105.
  • Found 669 failed-auth events, all targeting **root**.
  • No successful logins or other indicators of compromise.

3. **Action**
  • Applied 'block_ip' to IP **192.168.56.105** on both Ubuntu (iptables) and Windows (ssh).
  • Block persists for **60 seconds** as specified.

No escalation was needed – the policy's escalation rule does not apply here. The source IP has now been neutralized, preventing further brute-force attempts.

```

Figure 3: Agent reasoning trace for the SSH brute-force scenario.

4.10 Scenario 2: Escalation and Aggregated Alert Handling

In this scenario, the same Kali VM generated a higher-severity event pattern. Rule 40112 (Multiple authentication failures followed by a success) fired at severity HIGH (level 12), indicating a potential compromise. The agent's aggregation mechanism grouped six related alerts from the same source into a single aggregated alert, reducing processing overhead while preserving full evidentiary context. The agent identified the alert as a known test threat, automatically blocked the source IP, and created a CRITICAL priority escalation ticket (ESC-20260502-213341-702c) delivered to the human escalation channel. Figure 5 presents the terminal output for this scenario. The output confirms: "Incident escalated. Ticket: ESC-20260502-213341-702c" and "Auto-blocked and escalated." The system also correctly applies deduplication: subsequent alerts from the already-blocked IP are skipped with the message "IP 192.168.56.105 already blocked recently, skipping auto-block." This demonstrates that the agent correctly differentiates scenarios requiring autonomous action only (Scenario 1) from those requiring both autonomous action and human notification (Scenario 2).

contextual reasoning is not achievable with rule-based SOAR systems, which execute fixed playbooks regardless of contextual nuances such as the presence or absence of a successful login following a brute-force sequence.

Cross-Platform Autonomous Response: The `block_ip` tool executed containment actions on both the Ubuntu VM (iptables) and the Windows VM (netsh via SSH/paramiko) from a single agent decision. This unified cross-platform response capability eliminates the need for separate SOAR connectors or manual operator intervention on each platform, directly addressing the operational complexity identified in Section 2.7.

Intelligent Escalation with Full Auditability: The agent correctly differentiated between alerts requiring only autonomous response (Scenario 1: brute force with no successful login) and those requiring both autonomous response and human escalation (Scenario 2: Rule 40112, multiple failures followed by success). Every decision, tool invocation, and observation is logged in the SQLite audit database, producing the transparent “Glass Box” reasoning trail described in the research gap analysis (Table 4), and providing the interpretability identified by Jones (2025) [6] as a critical requirement for production deployment of autonomous security agents.

Taken together, these results confirm that the implemented system addresses all three dimensions of the research gap identified in Section 2.7.1: it simultaneously reasons about and executes responses; it provides explicit, auditable reasoning traces for every decision; and it interprets natural-language security policies in real time and maps them to concrete actions. These capabilities are not present in any single existing open-source system reviewed in the literature.

4.12 Conclusion

The experimental results validate the core hypothesis: a single ReAct agent can autonomously perform the triage, investigation, and response functions of a Tier-1 SOC analyst. The agent successfully blocks SSH brute-force attacks within 60 seconds and correctly escalates confirmed compromises with full audit trails. These results confirm that the system addresses all three dimensions of the research gap.

Conclusion

This project has presented the design, implementation, and evaluation of an Agentic AI Security Supervisor that functions as an autonomous Tier-1 SOC analyst. By integrating a ReAct (Reasoning + Acting) agent with the Wazuh SIEM platform, the system automatically ingests, normalises, enriches, and triages security alerts, then executes response actions without human intervention for a defined set of incident types. The architecture combines a four-stage autonomous triage engine (context enrichment, false-positive assessment, priority scoring, escalation pre-decision) with an LLM-powered investigation layer that follows explicit Thought-Action-Observation loops. Every decision, tool call, and observation is logged to an immutable SQLite audit trail, providing full transparency and post-incident verifiability.

Experimental results obtained from a virtual laboratory environment (Ubuntu, Windows, Kali) demonstrate that the agent autonomously detects and blocks SSH brute-force attacks across both Linux and Windows within 60 seconds – matching the speed of a human Tier-1 analyst while eliminating manual error. When a compromise is confirmed (multiple failures followed by a successful login), the agent correctly escalates the incident to a human Tier-2 analyst with a structured ticket containing a complete reasoning trace. The system successfully handles the full Tier-1 analyst workload: triage, investigation, decision, response, and escalation.

The main contributions of this work are threefold. First, it provides a concrete, open-architecture implementation of a ReAct-based security agent that bridges the gap between passive LLM assistants and fully autonomous response. Second, it introduces a transparent, evidence-driven triage engine that calculates false-positive probability and priority using weighted, interpretable rules. Third, it demonstrates cross-platform containment (Linux iptables and Windows netsh) from a single agent decision, eliminating the need for separate SOAR connectors or manual intervention.

Compared to recent work by Bonfanti et al. [4], which generates offline firewall rules from threat intelligence, the proposed supervisor executes real-time responses and maintains explicit reasoning traces. Compared to Jones’s [6] multi-agent identity threat hunting framework, this project achieves similar detection speed with a single ReAct agent and provides built-in auditability – a critical requirement for production deployment.

Nevertheless, the current implementation has several limitations. The system was tested only on SSH brute-force and privilege-escalation scenarios within a controlled virtual laboratory; its performance on other attack types (e.g., web application attacks, malware, insider threats) remains unvalidated. The false-positive and priority thresholds were empirically tuned on a limited dataset (approximately 50 simulated alerts) and may require recalibration for different organisational environments. The agent currently relies on a single LLM (GPT-OSS) and a fixed set of tools; more complex or unfamiliar attacks may exceed its reasoning capacity.

Future work will focus on expanding the tool set to cover additional response actions (e.g., blocking on network firewalls, isolating containers), integrating with other SIEM platforms (e.g., Splunk, QRadar) and threat intelligence feeds, and implementing a feedback loop that allows human analysts to correct or annotate agent decisions for continual learning.

Longer-term directions include fine-tuning the LLM on security-specific reasoning traces, experimenting with multi-agent coordination for advanced persistence detection, and formally evaluating the system on live SOC data to measure reduction in mean time to respond (MTTR) and analyst workload.

In summary, the Agentic AI Security Supervisor demonstrates that a single ReAct agent, guided by transparent triage and policy-driven tools, can effectively replace a human Tier-1 analyst for routine incident handling while maintaining full auditability. This work contributes a practical, extensible blueprint for autonomous security operations and opens the door to further research on trustworthy, human-aligned AI agents in critical infrastructure protection.

References

- [1] Kshetri, Nir. "Transforming cybersecurity with agentic AI to combat emerging cyber threats." *Telecommunications Policy* 49.6 (2025): 102976.
- [2] Ghosh, Shaona, et al. "A Safety and Security Framework for Real-World Agentic Systems." *arXiv preprint arXiv:2511.21990* (2025).
- [3] Singh, Y., Patel, N. D., & Shandilya, S. K. (2024). Enhancing Security Operations Center Efficiency through Multi-Model Integration of Large Language Models and SIEM Systems.
- [4] Bonfanti, Chiara, et al. "From Threat Intelligence to Firewall Rules: Semantic Relations in Hybrid AI Agent and Expert System Architectures." *arXiv preprint arXiv:2603.03911* (2026).
- [5] Sumers, Theodore, et al. "Cognitive architectures for language agents." *Transactions on Machine Learning Research* (2023).
- [6] Jones, Philip. "Agentic AI for Real-Time Threat Hunting: Automating Breach Detection in Enterprise Identity Systems."
- [7] Stefanov, M., et al. (2025). "Autonomous Agentic AI Architectures for Optimizing Security Operations Centers (SOC) KPIs: Methodology, Impact on Detection, Response, and Recovery." *Land Forces Academy Review*, Vol. XXX, No. 3(119).
- [8] Olabiyi W., "Combating Alert Fatigue: AI-Enhanced SIEM Framework for Optimized Incident Response" April 20
- [9] Veluru S.P. & Manchala M.K., "Using LLMs as Incident Prevention Copilots in Cloud Infrastructure" December 2024.

- [10] Tellache, A., Amara Korba, A., Mokhtari, A., Moldovan, H., & Ghamri-Doudane, Y. (2025). Advancing autonomous incident response: Leveraging LLMs and cyber threat intelligence. arXiv preprint arXiv:2508.10677.
- [11] Kim, M., Wang, J., Moore, K., Goel, D., Wang, D., Mohsin, A., Ibrahim, A., Doss, R., Camtepe, S., & Janicke, H. (2025). CyberAlly: Leveraging LLMs and knowledge graphs to empower cyber defenders. In Companion Proceedings of the ACM Web Conference 2025 (WWW Companion '25). ACM.
- [12] Naraduhita, B. P. P., Bumbungan, M. T. E., et al. (2025). Pemanfaatan LLM dalam integrasi Wazuh dan Jira untuk deteksi serangan dan rekomendasi respons otomatis [Leveraging LLM integration with Wazuh and Jira for automated cyberattack detection and incident response]. [Undergraduate Thesis, Institut Teknologi Sepuluh Nopember].
- [13] Wei, Bowen, et al. "CORTEX: Collaborative LLM Agents for High-Stakes Alert Triage." arXiv preprint arXiv:2510.00311 (2025).
- [14] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. In International Conference on Learning Representations (ICLR).